Software Architecture & Design (SAD) Specification

File Compression Tool — Huffman Coding, Python, `.hfc` / HFC1 Format

Field	Value
Document	Software Architecture & Design (SAD) Specification
Product	File Compression Tool
Version	1.0
Date	6 November 2026
Format	IEEE format, as required by `02TheCheckList.pdf`
Repository / project	`File_Compression_Tool_BLs` (`file-compression-tool-main`)
Implementation language	Python (standard library only, as implemented)
Compression method	Huffman coding, lossless
Compressed file format	`.hfc`, magic `HFC1`
Author (README)	Keerthana Kumaresh (GitHub: `KeerthanaKumaresh1`)
Team (usernames only)	`KeerthanaKumaresh1`, `rishi2908`, `khushi309`
Status	Draft — written from the implemented code in the supplied zip

> **Scope note for the reader / viva panel.** This SAD describes the system **as implemented** in the supplied source zip, not as originally planned. Where the code differs from the plan, the difference is stated in §10 (Deviations & Known Limitations) instead of being smoothed over. Anything that could not be verified from the evidence supplied is stated as not in the evidence, and must not be presented as fact in the demo or viva.
>
> **What this document does not contain.** No PR numbers, no GitHub Actions run identifiers, no video links, and no team full names beyond the one author name given in `README.md` are stated in this draft, because none of those were in the evidence supplied. The checklist (`02TheCheckList.pdf`) requires a 1-minute Sprint 1 video and a 2-minute Sprint 2 video and GitHub Actions / PR evidence; those artefacts were not in the evidence supplied — add their links / identifiers only from the real repository, do not invent them.

Revision History

Version	Date	Author / change	Description
1.0			

Version	Date	Author / change	Description
	6 Nov 2026	SAD draft from implemented code	Initial IEEE-format SAD: architecture, component design, data design, sequence diagrams, API design, error handling, security architecture, traceability, deviations. Written against `file-compression-tool-main` as extracted from `file-compression-tool-main__1.zip`.

Contents

- 1 Introduction
- 2 Architectural Overview
- 3 Component Design
- 4 Data Design
- 5 Sequence Diagrams
- 6 API Design
- 7 Error Handling Protocols
- 8 Security Architecture
- 9 Traceability
- 10 Deviations & Known Limitations
- Appendix A — Backlog Summary

---

1 Introduction

1.1 Purpose

This document describes the software architecture and detailed design of the File Compression Tool, a lossless file-compression application built around Huffman coding. It is the SAD deliverable required by the mini-project checklist (`02TheCheckList.pdf`, Part 1): IEEE format; architecture with component diagrams/descriptions, architectural pattern, traceability and security architecture; design section with at least two UML sequence diagrams, API design and error-handling protocols.

Its readers are the project team, the reviewer / viva panel, and any maintainer who needs to understand, test or change the code. A reader should be able to trace every backlog item (BL1–BL20) to a component, a requirement and a test file (§9), to reconstruct the `.hfc` file format from §4, and to explain any component's behaviour from §3 without reading the whole codebase first.

1.2 Scope

The system in scope is exactly what is in the supplied source tree:

- **Entry point:** `main.py` — a thin entry point that calls `compressor.cli.main()` and exits with its return code.
- **Core package:** `compressor/` — 21 modules (excluding `__init__.py`), implementing frequency analysis, Huffman tree/code construction, the `.hfc` file format, compression and

decompression, validation, statistics, verification, error handling, output protection, performance measurement, history and batch compression.
- **Graphical interface:** `gui.py` (785 lines) — a five-tab Tkinter desktop interface.
- **Tests:** `tests/` — `unittest` tests. Verified run: **81 tests, all passed, 0.28–0.62 s**.
- **Documentation already in the repository:** `documents/` — `SRS_document.pdf`, `Security_Requirements.pdf`, `Test_Cases.pdf`, `Use_Case_Flow.pdf`, `User_Stories.pdf`; and `README.md`.

Out of scope for this SAD: any network service, any server, any file format other than `.hfc` / HFC1, and any compression algorithm other than Huffman coding. The tool is a local, standalone, single-machine application (see §8, SR-9).

1.3 References

#	Document / artefact	What it is used for in this SAD
R1	`01FAQs_MiniProjectDemo.pdf`	Demo / viva expectations; deviations must be documented and explained (§10).
R2	`02TheCheckList.pdf`	SAD checklist requirements (IEEE format; component diagrams, pattern, traceability, security architecture; ≥ 2 sequence diagrams, API design, error handling).
R3	`documents/SRS_document.pdf`	Functional requirements FR-1–FR-10 and non-functional requirements NFR-1–NFR-8 (§9).
R4	`documents/Security_Requirements.pdf`	Security requirements SR-1–SR-10 (§8).
R5	`documents/Test_Cases.pdf`, `documents/Use_Case_Flow.pdf`, `documents/User_Stories.pdf`	Supporting requirements / test material in the repository; their contents were read and are used in the SRS (FRs, SRs, UT_01–UT_10).
R6	`README.md` (in the zip)	Project objectives, repository structure, author.
R7	Source code in `file-compression-tool-main/` (`main.py`, `gui.py`, `compressor/*.py`, `tests/*.py`)	Primary evidence for every design statement in §§2–8. Where code and plan disagree, the code is described and the disagreement is logged in §10.
R8	Backlog screenshot (`File_Compression_Tool_BLs`, GitHub Projects, Backlog view)	BL1–BL20, assignees, story points, sprint assignment, all `Done` (Appendix A).

1.4 Definitions, Acronyms and Abbreviations

Term	Meaning
Huffman coding	A lossless compression method that assigns shorter bit strings to more frequent byte values and longer bit strings to rarer ones, using a binary tree built from symbol frequencies.
`.hfc` / HFC1	The tool's own compressed-file format. Files end in `.hfc`; every file starts with the 4-byte magic `HFC1`. See §4.1.
CRC32	

Term	Meaning
	Cyclic Redundancy Check, 32-bit (Python `zlib.crc32`). Stored in the `.hfc` header over the original data and checked on decompression. Detects accidental corruption; it is not a cryptographic checksum.
SHA-256	Secure Hash Algorithm, 256-bit (Python `hashlib.sha256`). Used only by the verification module to compare original and restored files byte-for-byte; it is not stored in the `.hfc` file.
Frequency table	A mapping `{byte value (0–255): count}` for one input file, stored in the `.hfc` file so the decoder can rebuild the same Huffman tree.
Huffman tree	A binary tree whose leaves are byte values. Left edge = bit `0`, right edge = bit `1`. The path from the root to a leaf is that byte's code.
Min-heap	A priority queue in which the smallest-priority item is removed first. Used to build the Huffman tree by repeatedly combining the two lowest-frequency nodes.
Padding	The number of unused bits (0–7) in the last byte of the compressed payload, stored in the header so the decoder knows where the real bits end.
BL*n*	Backlog item *n* (BL1–BL20), as shown in the GitHub Projects backlog (Appendix A).
FR-*n* / NFR-*n* / SR-*n*	Functional / non-functional / security requirement numbers from the SRS and Security Requirements documents (R3, R4).
CLI / GUI	Command-line interface (`compressor/cli.py`, via `main.py`) / graphical interface (`gui.py`, Tkinter).
Pipeline	`compressor/pipeline.py` — the module that orchestrates the full compress / decompress workflow for the CLI.

1.5 Verified Behaviour Used as Design Evidence

The following numbers are the verified measurements supplied with this task. They are used exactly as given; they are demo measurements, not performance guarantees.

- **Live demo, repetitive text:** a 90,000-byte repetitive text file compressed to **50,914 bytes** — **43.43% saved, ratio 1.77:1, 0.107 s**; verification passed and the decompressed file was byte-identical (`cmp`).
- **Small-file overhead:** a 10-byte file compressed to **42 bytes** — larger than the original, because the 19-byte header plus the frequency table outweigh the payload saving. See §10.
- **Tests:** 81 tests, all passed, 0.28–0.62 s.
- **Coverage (compressor package):** line coverage **83% (641 statements, 106 missed)**; branch coverage **79% (226 branches, 35 partial)**. `corruption.py` **0%**; `cli.py` **62% line / 50% branch**; `file_validation.py` **65%**; `gui.py` **not covered** (not imported by tests).

---

2 Architectural Overview

2.1 Architectural Pattern — Layered / Modular Pipeline

The architecture is a **layered, modular pipeline**:

1. **Presentation layer (thin).** Two interchangeable front ends — the CLI (`compressor/cli.py`, reached through `main.py`) and the GUI (`gui.py`). Neither front end contains compression logic of its own; each one collects input from the user, calls the core package, and presents results.
2. **Orchestration layer.** `compressor/pipeline.py` (`compress_file()`, `decompress()`) sequences the core steps for the CLI path. `compressor/batch_compression.py` does the same, in a simpler form, for multiple files.
3. **Core / domain layer.** Leaf modules that are, wherever possible, pure functions over explicit inputs: frequency counting, heap, tree, codes, file format, decompression, statistics, verification. Each leaf module does one job and can be tested on its own — which is why most of the test suite targets leaf modules directly.
4. **Cross-cutting services.** Validation, output protection, error handling, history and performance measurement, used by the orchestration layer (and, in part, by the GUI).

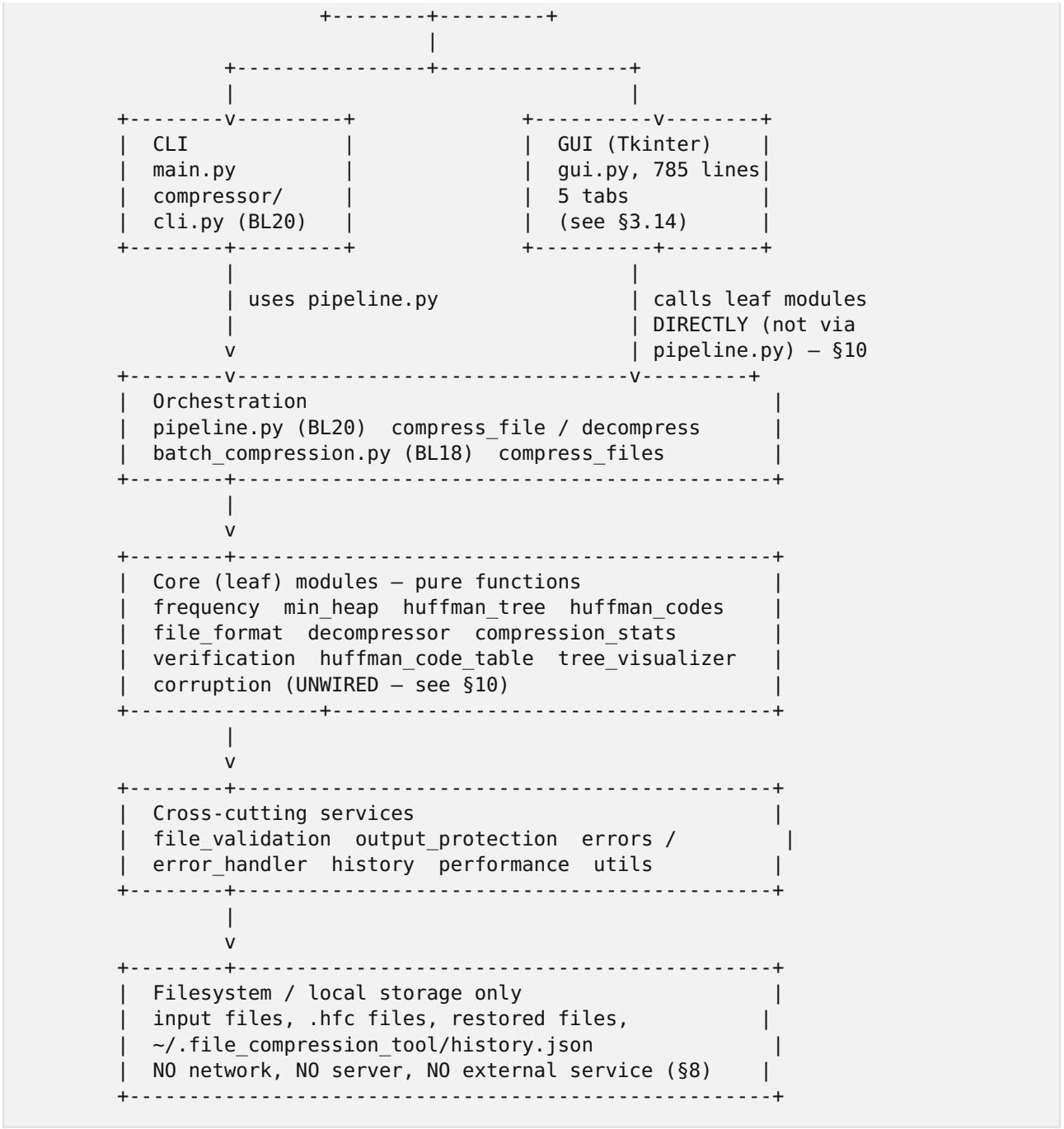
Why this pattern fits.

- **The algorithm is a pipeline.** Compression is a fixed sequence — count frequencies → build tree → generate codes → encode → write — and decompression is its reverse. A pipeline module that calls leaf functions in order is the most direct expression of that sequence, and it is exactly what `pipeline.py` does.
- **Two front ends share one core.** The CLI and the GUI both need the same Huffman machinery. Putting that machinery in `compressor/` and keeping the front ends thin avoids duplicating the algorithm.
- **Testability.** Leaf modules are pure functions (`calculate_frequencies(data)` takes bytes and returns a dict; `generate_huffman_codes(root)` takes a tree and returns a dict). They can be unit-tested without files, a GUI, or argument parsing. This is reflected in the test suite, which is organised one test file per leaf module.
- **Determinism.** The decoder rebuilds the Huffman tree from the frequency table stored in the file; it does not store the tree or the codes. That only works if tree construction is deterministic for a given frequency table. Determinism is therefore a design rule, enforced in `huffman_tree.py` (sorted insertion) and `min_heap.py` (insertion-order tie-breaking) — see §3.3.

Pattern limits, stated honestly. The pattern is only partly realised in the GUI path: the GUI does **not** use `pipeline.py` (see §2.3 and §10). There is therefore one orchestrated path (CLI → pipeline) and one hand-wired path (GUI → leaf modules directly). That is a deviation to fix, not a second pattern.

2.2 Component Diagram





Data flows in one direction during compression (top → bottom in the diagram, ending in a `.hfc` file) and in the reverse direction during decompression (`.hfc` file → restored file). History is the only side-effect besides the output file itself: the pipeline records each operation in `history.json`; the GUI keeps a separate, session-only history in memory (see §3.12).

2.3 Component Table

#	Component / module	BL	Responsibility	Depends on
1	`main.py`	— (skeleton)	Thin entry point: `sys.exit(cli.main())`.	`compressor.cli`
2	`compressor/cli.py`	BL20	Argument parsing; `compress` / `decompress` / `verify` /	`pipeline`, `compression_stats`,

#	Component / module	BL	Responsibility	Depends on
			`history` subcommands; prints statistics, code table, tree, timing.	`error_handler`, `huffman_code_table`, `performance`, `tree_visualizer`, `utils`, `verification`, `history`
3	`gui.py`	— (no BL number assigned in the backlog screenshot)	Five-tab Tkinter interface; file pickers, result display, session history/performance.	Leaf modules directly: `frequency`, `huffman_tree`, `huffman_codes`, `file_format`, `decompressor`, `tree_visualizer`, `huffman_code_table`, `compression_stats`
4	`compressor/pipeline.py`	BL20	Orchestrates one-file compress and decompress end to end; records history.	`file_validation`, `file_format`, `frequency`, `huffman_tree`, `huffman_codes`, `output_protection`, `compression_stats`, `decompressor`, `history`
5	`compressor/batch_compression.py`	BL18	Compresses a list of files, one after another; returns per-file results.	`file_validation`, `frequency`, `huffman_tree`, `huffman_codes`, `file_format`, `output_protection`
6	`compressor/frequency.py`	BL3	Counts byte frequencies, sorted by byte value.	— (pure function)
7	`compressor/min_heap.py`	BL4	Hand-written array min-heap with deterministic tie-breaking.	—
8	`compressor/huffman_tree.py`	BL5	`HuffmanNode`; builds the Huffman tree from a frequency table.	`min_heap`
9	`compressor/huffman_codes.py`	BL6	Generates prefix codes from a tree (left `0`, right `1`; single symbol → `"0"`).	— (operates on a tree)
10	`compressor/file_format.py`	BL8	`.hfc` format: header, frequency table, payload encoding, writing with partial-output cleanup.	`errors`
11	`compressor/decompressor.py`	BL9		`file_format`, `file_validation`,

#	Component / module	BL	Responsibility	Depends on
			Parses `.hfc`, rebuilds the tree, decodes the payload, checks CRC32.	`huffman_tree`, `errors`
12	`compressor/verification.py`	BL10	SHA-256 file hash; byte-for-byte lossless verification.	`decompressor`, `file_validation`, `errors`
13	`compressor/compression_stats.py`	BL11	Space saved, percentage saved, compression ratio, average bits per symbol; formatted display.	—
14	`compressor/tree_visualizer.py`	BL12	Renders a Huffman tree as indented text (max depth 8).	—
15	`compressor/huffman_code_table.py`	BL13	Builds the frequency/code table rows for display.	—
16	`compressor/output_protection.py`	BL14	Default output names; overwrite protection; exclusive-create write.	`file_format` (for `EXTENSION`), `errors`
17	`compressor/corruption.py`	BL15	Corruption checks: `validate_structure`, `check_payload_length`, `check_checksum`. **Not called by any production code — see §10.**	`errors`
18	`compressor/performance.py`	BL16	Timing (`time.perf_counter`) and optional peak-memory measurement (`tracemalloc`); `PerformanceReport`.	`utils`
19	`compressor/error_handler.py`	BL17	`run_safely`: catches errors, prints one-line messages, returns exit codes.	`errors`
20	`compressor/errors.py`	— (skeleton)	Exception hierarchy: base `CompressionError` + 5 subclasses.	—
21	`compressor/history.py`	BL19	Persistent JSON history (`~/.file_compression_tool/history.json`, max 100 entries).	— (stdlib `json`, `pathlib`, `datetime`)
22	`compressor/file_validation.py`	BL2	Input validation and whole-file reading.	`errors`
23	`compressor/utils.py`	— (skeleton)	Human-readable file sizes (`format_size`).	—

Notes on the table: BL1 (File Selection) has no dedicated module — file selection is the CLI's `input` argument and the GUI's file picker. BL7 (Bitstream Encoding) has no dedicated module either — bit-packing lives inside `file_format.encode_to_bytes()` and bit-walking inside

`decompressor.decode_payload()`; see §3.5 and §3.6. The backlog screenshot assigns no BL number to `gui.py` or to `main.py` / `errors.py` / `utils.py`; those are recorded as "—" rather than inventing a number.

---

3 Component Design

Each subsection gives the component's purpose, its public interface (exact function / class names from the code), its behaviour including edge cases, and what depends on it. Signatures are quoted from the source; where a signature has no type annotations in the code, none are claimed here.

3.1 Entry Point — `main.py`

- **Purpose.** The only thing a user runs from the command line (besides `gui.py`). Keeps startup trivial so that all behaviour lives in testable modules.
- **Interface.** No functions of its own. On execution it runs `sys.exit(main())`, where `main` is `compressor.cli.main`.
- **Behaviour.** The exit code is whatever `cli.main()` returns via `run_safely` (see §7). A normal run returns `0`.

3.2 File Selection (BL1) — No Dedicated Module

- **Purpose.** Let the user choose the input file (FR-1).
- **CLI.** The `input` positional argument of `compress` / `decompress`; the `original` and `compressed` arguments of `verify`.
- **GUI.** `filedialog.askopenfilename(...)` in `choose_compress_file()` / `decompress_file()`; `filedialog.asksaveasfilename(...)` for the output, with default extension `.hfc` for compression.
- **Design point.** Selection and validation are separate steps: selection only produces a path; `file_validation.validate_input_file()` decides whether the path is usable (§3.10). The GUI additionally refuses to let the output path equal the input path (`os.path.abspath(source) == os.path.abspath(output)` → error message box), so the original can never be overwritten by its own compressed output.

3.3 Frequency Analysis — `compressor/frequency.py` (BL3, FR-2)

- **Interface.** `calculate_frequencies(data) -> {byte: count}`, sorted by byte value.
- **Behaviour.** Iterates over the input bytes once, counting occurrences with a dict. Returns `dict(sorted(freq.items()))` — sorting matters: it makes the frequency table, the tree construction order and therefore the generated codes deterministic.
- **Edge cases.** Empty input returns `{}` (an empty dict). The caller is responsible for rejecting empty files before this point (the pipeline does so via `validate_input_file`).
- **Complexity / limits.** Whole input is held in memory as `bytes`; see §10.

3.4 Min-Heap — `compressor/min_heap.py` (BL4)

- **Purpose.** Priority queue for Huffman tree construction. Hand-written array heap — `heapq` is deliberately not used (the module docstring says so).
- **Interface.** `MinHeap()` with `push(priority, item)`, `pop() -> (priority, item)`, `peek() -> (priority, item)`, `__len__`, `is_empty()`. Internal: `_sift_up`, `_sift_down`, `_swap`, `_key`.
- **Representation.** `_data` is a list of `(priority, order, item)` tuples; `_counter` is a monotonically increasing insertion counter.
- **Determinism rule (important for the decoder).** Ties in priority are broken by insertion order (`_key` returns `(priority, order)`). Equal-frequency symbols therefore always come out in the order they were pushed, so the tree built during decompression — from the same frequency table, inserted in sorted-symbol order — is identical to the tree built during compression.
- **Errors.** `pop()` / `peek()` on an empty heap raise `IndexError("pop from an empty heap")` / `IndexError("peek at an empty heap")`.

3.5 Huffman Tree — `compressor/huffman_tree.py` (BL5, FR-3)

- **Interface.**
- `class HuffmanNode(freq, symbol=None, left=None, right=None)` with attributes `freq`, `symbol`, `left`, `right`, and method `is_leaf()` (true when both children are `None`).
- `build_huffman_tree(freq) -> root | None`.
- `symbol_label(symbol) -> str` — readable labels: `\t`, `\n`, `\r`, `space` for 9, 10, 13, 32; printable ASCII as itself; anything else as `0xNN`.
- **Algorithm.** If `freq` is empty, return `None`. Otherwise push one leaf node per symbol onto a `MinHeap`, in sorted-symbol order (deterministic tie-breaking, §3.4). While more than one node remains, pop the two lowest-frequency nodes, create a parent with `freq = f1 + f2`, `symbol = None`, `left = first`, `right = second`, and push the parent back. Return the single remaining node.
- **Edge cases.** One distinct symbol → the tree is a single leaf; the code for that symbol is `"0"` (handled in §3.6, not by a special tree shape). Empty frequency table → `None`, and the caller must handle it (`generate_huffman_codes(None)` returns `{}`; `decompressor` raises `CorruptedFileError` for an empty table in a non-empty file).

3.6 Huffman Codes — `compressor/huffman_codes.py` (BL6, FR-4)

- **Interface.** `generate_huffman_codes(root) -> {symbol: code-string}`.
- **Behaviour.** Depth-first traversal from the root. Left edge appends `"0"`, right edge appends `"1"`. A leaf's accumulated prefix is its code. A tree with a single symbol assigns that symbol the code `"0"` (`prefix or "0"`).
- **Properties.** Codes are prefix codes: no code is a prefix of another, so decoding by walking the tree bit by bit is unambiguous. An empty tree (`root is None`) returns `{}`.
- **Bitstream encoding (BL7).** Encoding itself is not a separate module. The forward direction — packing code bits into bytes — is `file_format.encode_to_bytes(data, codes) -> (payload_bytes, padding_bits)`: it accumulates bits in an integer buffer, emits a byte each time 8 bits are available, and pads the final partial byte with zero bits on the right, returning the padding count (0–7). The reverse direction — walking the tree bit by bit — is `decompressor.decode_payload()` (§3.7).

3.7 Compressed File Creation — `compressor/file_format.py` (BL8, FR-5, FR-6)

- **Interface / constants.**
- `MAGIC = b"HFC1"`, `HEADER_FORMAT = ">4sQIBH"`, `HEADER_SIZE = 19` bytes, `ENTRY_FORMAT = ">BI"`, `ENTRY_SIZE = 5` bytes, `EXTENSION = ".hfc"`, `MAX_FILE_SIZE = 0xFFFFFFFF` (4,294,967,295 bytes; a code constant — see §4.1).
- `encode_to_bytes(data, codes) -> (payload_bytes, padding_bits)`.
- `build_compressed_bytes(data, freq, codes) -> bytes` — the complete `.hfc` file in memory.
- `write_bytes(path, data)` — writes bytes; on write failure removes the partial output and raises `FileCreationError`.
- **Behaviour of `build_compressed_bytes`.** Raises `FileCreationError("File is too large to compress (limit is 4 GB).")` if `len(data) > MAX_FILE_SIZE`. Otherwise: encode the payload (`encode_to_bytes`); pack the header with `struct.pack(HEADER_FORMAT, MAGIC, len(data), zlib.crc32(data), padding, len(freq))`; append one 5-byte entry per distinct symbol, in sorted-symbol order (`struct.pack(ENTRY_FORMAT, symbol, freq[symbol])`); return `header + table + payload`. All numeric fields are big-endian.
- **Behaviour of `write_bytes`.** Opens the path for writing; if opening fails, raises `FileCreationError("Could not create ...")`. If writing fails, it attempts `os.remove(path)` (ignoring failure of the removal itself) and raises `FileCreationError("Could not write ...")`. This is the SR-10 partial-output cleanup (see §8).
- **Design point.** The frequency table, not the tree or the codes, is stored in the file. That is sufficient to rebuild the tree (deterministically, §3.4) and keeps the format simple and self-describing. The price is the per-symbol table overhead — see the 10-byte → 42-byte example in §1.5.

3.8 File Decompression — `compressor/decompressor.py` (BL9, FR-7)

- **Interface.**
- `parse_compressed(raw) -> (original_size, crc, padding, freq, payload)`.
- `decode_payload(payload, root, original_size, padding) -> bytes`.
- `decompress_bytes(raw) -> bytes` — parse, check, rebuild tree, decode, verify CRC32.
- `decompress_file(input_path, output_path) -> restored_size (int)`.
- **Behaviour of `parse_compressed` (validation first, in this order).** Rejects the input with `CorruptedFileError` if: it is shorter than the 19-byte header (`"File is too short to be a compressed file."`); the magic is not `HFC1` (`"Not a valid compressed file (wrong signature)."`); `padding > 7`; the file ends before the frequency table ends (`"The frequency table is incomplete."`); the table contains a duplicate symbol; or any frequency is zero. Otherwise returns the five parts.
- **Behaviour of `decode_payload`.** Empty original (`original_size == 0`): returns `b""` only if there is no payload and no padding, otherwise `CorruptedFileError`. Single-symbol tree (`root.is_leaf()`): returns that byte repeated `original_size` times, but only if there is no payload and no padding, otherwise `CorruptedFileError`. General case: walks the tree bit by bit (`payload[i >> 3] >> (7 - (i & 7))) & 1`, `0` = left, `1` = right), appending a byte each time a leaf is reached and returning to the root. At the end it requires the walk to be back at the root and `len(out) == original_size`, otherwise `CorruptedFileError("The data does not decode to the expected size.")`.

- **Behaviour of `decompress_bytes`.** Parses, then checks `sum(freq.values()) == original_size` (raises `CorruptedFileError` if not), rebuilds the tree with `build_huffman_tree(freq)`, decodes, and finally checks `zlib.crc32(restored) == crc` (raises `CorruptedFileError("Corrupted file: CRC32 checksum mismatch.")` if not). Only a fully validated, checksum-matching result is returned.
- **Behaviour of `decompress_file`.** Reads the input with `read_file_bytes`, decompresses in memory with `decompress_bytes`, writes the result with `write_bytes`, and returns `len(data)`.

3.9 Lossless Verification — `compressor/verification.py` (BL10, FR-10, SR-4)

- **Interface.**
- `file_hash(path) -> hex-digest string` — SHA-256, read in 1 MiB chunks so a large file is not loaded for hashing. Raises `FileValidationError` if the file cannot be read.
- `files_identical(path_a, path_b) -> bool` — compares SHA-256 digests.
- `verify_lossless(original_path, compressed_path) -> bool` — decompresses the compressed file in memory (`decompress_bytes(read_file_bytes(compressed_path))`) and compares its SHA-256 with the original's.
- `assert_lossless(original_path, compressed_path)` — raises `VerificationError("Verification FAILED: the restored data differs from the original.")` if `verify_lossless` is false.
- **Design point.** Verification is a *second, independent* integrity check on top of the CRC32 inside the format: CRC32 is stored in the file and checked on every decompression; SHA-256 verification re-decompresses and compares against the original file, and is only run when the user asks for it (`--verify` in the CLI). It is also the mechanism behind the verified demo result in §1.5.

3.10 File Validation — `compressor/file_validation.py` (BL2, SR-1)

- **Interface.**
- `validate_input_file(path, allow_empty=False) -> path` — raises `FileValidationError` (see §7 for messages).
- `read_file_bytes(path) -> bytes`.
- **Checks, in order.** No path / blank path → error. Path does not exist → `"File not found: ..."`. Path is a directory → `"... is a folder, not a file."`. Path is not readable (`os.access(path, os.R_OK)`) → `"No permission to read ..."`. File is empty and `allow_empty` is false (the default) → `"... is empty - there is nothing to compress."`. The function returns the path unchanged on success; it does not open the file.
- **`read_file_bytes`.** Opens the path in binary mode and returns `handle.read()` — the whole file in memory. On `OSError` it raises `FileValidationError("Could not read ...")`. Note the contrast with `file_hash` (§3.9), which streams: validation/reading is whole-file, hashing is chunked. See §10.

3.11 Compression Statistics — `compressor/compression_stats.py` (BL11, FR-8)

- **Interface.**
- `calculate_statistics(original_size, compressed_size, total_bits=None) -> dict` with keys `original_size`, `compressed_size`, `space_saved`, `space_saved_percent` (rounded to 2

dp), `compression_ratio` (rounded to 2 dp, or `None`), `average_bits_per_symbol` (rounded to 2 dp, or `None`).

- `format_statistics(statistics) -> dict` of display strings: `"Original size"`, `"Compressed size"`, `"Space saved"`, `"Space saved percentage"`, `"Compression ratio"` (`"N/A"` when `None`), `"Average bits per symbol"` (`"N/A"` when `None`).
- **Behaviour / edge cases.** `space_saved = original_size - compressed_size` (negative when the compressed file is larger — the small-file case). `space_saved_percent = space_saved / original_size * 100`. `compression_ratio = original_size / compressed_size` (or `None` when the compressed size is 0). `average_bits_per_symbol = total_bits / original_size` when `total_bits` is given and the original is non-empty, else `None`. Original size 0 → saved 0, 0.0%, ratio `None`. Non-integer sizes, boolean sizes, negative sizes and negative `total_bits` raise `TypeError` / `ValueError` as coded.
- **Note.** The pipeline passes `total_bits` (computed as `sum(frequencies[s] * len(codes[s]))`), so CLI statistics include average bits per symbol. The GUI calls `calculate_statistics(len(data), len(compressed))` without `total_bits`, so its statistics show `"N/A"` for average bits per symbol — a minor inconsistency between the two front ends (see §10).

3.12 Huffman Tree Visualisation & Code Table — `tree_visualizer.py` (BL12), `huffman_code_table.py` (BL13) (FR-9)

- **`render_tree(root, max_depth=8) -> str`.** Returns an indented text diagram. The root line is its label; each child is shown as `├── 0: <label>` / `└── 1: <label>`, with `│` / four-space continuation prefixes. Leaf labels are `'symbol' (frequency)`; internal nodes are `(frequency)`. Rendering stops at `max_depth` with a `└── ...` line. An empty tree renders as `"(empty tree)"`; a negative `max_depth` raises `ValueError`. The renderer accepts either `HuffmanNode` objects or dicts with the same attributes, and its symbol labelling differs in detail from `huffman_tree.symbol_label` (printable ASCII shown as-is, otherwise `0xNN`) — the two labellers are not shared code.
- **`build_huffman_code_table(frequencies, codes) -> [row-dict, ...]`.** One row per symbol, sorted by byte value, each row `{"byte": int, "display": repr(bytes([symbol])), "frequency": int, "code": str}`. Validates its inputs strictly: non-mappings → `TypeError`; mismatched symbol sets → `ValueError`; symbols outside 0–255, negative frequencies or non-string codes → `ValueError` / `TypeError`.
- **Use.** The CLI prints the table with `--table` and the tree with `--tree`; the GUI's Huffman Explorer tab shows both for a chosen file.

3.13 Output & Overwrite Protection — `compressor/output_protection.py` (BL14, SR-3, SR-7)

- **Interface.**
- `default_compressed_name(input_path) -> str` — input path + `".hfc"`.
- `default_decompressed_name(compressed_path) -> str` — strips a trailing `.hfc`, then inserts `"_restored"` before the remaining extension (e.g. `notes.txt.hfc` → `notes_restored.txt`).
- `prepare_output_path(output_path, overwrite=False) -> str` — validates an output path.
- `write_output(output_path, data, overwrite=False) -> str` — validates, then writes.

- **Behaviour of `prepare_output_path`.** Rejects a non-boolean `overwrite` with `TypeError`. Rejects an output path that is a directory (`FileCreationError`). If the path exists and `overwrite` is false, raises `OutputExistsError("Output file already exists: ... Choose another output path or enable overwrite.")` — the existing file is **not** modified. If the parent directory does not exist, raises `FileCreationError("Output directory does not exist: ...")`.
- **Behaviour of `write_output`.** Re-validates via `prepare_output_path`, then writes with mode `"xb"` (exclusive creation) when `overwrite` is false — so a file created between the check and the write still cannot be silently replaced; that race raises `FileExistsError`, converted to `OutputExistsError`. With `overwrite=True` it writes with `"wb"`. An `OSError` becomes `FileCreationError`.
- **Design point.** Protection is *reject-by-default*: overwriting requires the explicit `--overwrite` flag (CLI) or an explicit choice (GUI save dialog). There is no interactive "are you sure?" prompt in the CLI; the flag is the authorisation. This satisfies SR-3 as implemented ("requests confirmation or rejects the operation") on the rejection branch — see §8.

3.14 Corrupted File Detection — `compressor/corruption.py` (BL15, FR-10, SR-2, SR-5)

- **Interface (all three functions raise `CorruptedFileError` on failure, return `None` on success).**
- `validate_structure(original_size, padding, freq)` — padding must be ≤ 7; every frequency must be > 0; `sum(freq.values())` must equal `original_size`.
- `check_payload_length(freq, codes, padding, payload)` — the expected bit count is `sum(freq[s] * len(codes[s]))`; the payload must be exactly `(expected_bits + 7) // 8` bytes and the padding must equal `expected_bytes * 8 - expected_bits`.
- `check_checksum(data, expected_crc)` — `zlib.crc32(data)` must equal `expected_crc`.
- **Status — read this before the viva.** These three functions are **not called by `pipeline.py`, `decompressor.py`, the CLI or the GUI**, and they have **0% coverage** in the verified coverage run. Corruption detection in the running system is done inside `decompressor.py` instead (§3.8), which duplicates `validate_structure`'s and `check_checksum`'s checks inline and does not perform `check_payload_length`'s exact-length check at all. `test_corruption.py` tests corruption detection through `decompressor.decompress_bytes`, not through `corruption.py`. Full analysis in §10. Do not claim in the demo that `corruption.py` is wired into the decompression path.

3.15 Performance Measurement — `compressor/performance.py` (BL16, NFR-1, NFR-6)

- **Interface.**
- `@dataclass PerformanceReport(seconds: float, peak_memory: int = 0)`.
- `measure(func, *args, track_memory=False, **kwargs) -> (result, PerformanceReport)` — runs `func`, times it with `time.perf_counter()`, and, only when `track_memory` is true, starts `tracemalloc` and records peak traced memory; `tracemalloc` is always stopped in a `finally` block.
- `format_performance(report, size_bytes) -> str` — `"Time: X.XXX s | Speed: Y.YY MB/s"`, plus `" | Peak memory: ..."` only when `peak_memory` is non-zero.
- **Use.** The CLI wraps `compress_file` / `decompress` in `measure(..., track_memory=args.time)`; `--time` is therefore the switch that enables

memory tracking, and without it the report carries `peak_memory = 0`. The GUI does **not** use this module: it times its own operations with `time.perf_counter()` directly and computes throughput itself in the Statistics & Performance tab. The GUI's performance text explicitly warns that a single-operation timing "is not a controlled benchmark".

3.16 Error Handling — `compressor/errors.py`, `compressor/error_handler.py` (BL17, FR-10, SR-5, SR-8)

- **`errors.py`.** Base class `CompressionError(Exception)` and five subclasses: `FileValidationError`, `FileCreationError`, `CorruptedFileError`, `OutputExistsError`, `VerificationError`. Every error the system raises deliberately is one of these (or a subclass of them), so callers can handle "a problem with the tool's own operation" as one category.
- **`error_handler.py`.** `EXIT_CODES` maps the five subclasses to exit codes 2–6 (see §7). `exit_code_for(error)` returns the mapped code, or `1` for any other `CompressionError` (and for any other exception). `format_error(error)` returns the single-line string `f"Error: {error}"`. `run_safely(func, *args, **kwargs)` runs the function and returns an exit code instead of propagating: `0` on success; the mapped code for a `CompressionError` (message printed to stderr, no traceback); `130` for `KeyboardInterrupt` (`"Cancelled by the user."`); `7` for `MemoryError` (`"Error: not enough memory to process this file."`); `1` for any other exception (`"Unexpected error: {error}"`). See §7 for the full protocol.

3.17 Compression History — `compressor/history.py` (BL19)

- **Interface.** `class CompressionHistory(history_file=DEFAULT_HISTORY_FILE, max_entries=MAX_HISTORY_ENTRIES)` with `get_history() -> [entry, ...]`, `add_entry(operation, input_file, output_file, **details) -> entry`, `clear_history()`.
- **Storage.** `DEFAULT_HISTORY_FILE = Path.home() / ".file_compression_tool" / "history.json"`; `MAX_HISTORY_ENTRIES = 100`.
- **Entry schema.** Each entry is a JSON object: `timestamp` (UTC ISO-8601, `datetime.now(timezone.utc).isoformat()`), `operation` (`"compression"` or `"decompression"` — any other value raises `ValueError`), `input_file`, `output_file`, plus caller-supplied details (the pipeline passes `original_size` / `compressed_size` for compression and `restored_size` for decompression). See §4.2.
- **Behaviour.** `get_history()` returns `[]` if the file is missing, unreadable or not a JSON list, and drops any non-dict entries; it returns at most the last `max_entries` entries. `add_entry()` validates that the entry is JSON-serialisable before writing, appends, trims to the last 100, and writes atomically: write to a `.tmp` sibling file, then `replace()` it over the real file, removing the temporary file in a `finally` block. `clear_history()` writes `"[]"`. A non-integer or boolean `max_entries`, or one below 1, raises `TypeError` / `ValueError` at construction.
- **GUI difference.** The GUI does **not** use this module. It keeps `self.history = []` in memory for the current session only; its History tab states that the history "is held in memory and is cleared when the application closes". The CLI `history` subcommand reads the persistent file. The two histories are therefore different views of different data — see §10.

3.18 Batch Compression — `compressor/batch_compression.py` (BL18)

- **Interface.** `compress_files(input_paths, output_dir=None, overwrite=False) -> [result-dict, ...]`, where each result is `{"input_path", "output_path", "original_size", "compressed_size"}`.
- **Behaviour.** An empty input list raises `ValueError("Select at least one file to compress.")`. For each file, in order: validate, read, count frequencies, build tree, generate codes, build compressed bytes, choose the output path (`default_compressed_name`, or `output_dir` joined with that name), write with `write_output`. Results accumulate in a list; there is no early summary and no statistics calculation.
- **Design notes.** Batch compression re-implements the compression steps directly instead of calling `pipeline.compress_file()` — the same duplication pattern as the GUI (§10). It therefore does **not** record history entries and does **not** compute statistics. A failure part-way through (e.g. the third file is invalid) propagates the exception and stops the batch; files already compressed stay compressed — there is no rollback, and no per-file success/failure result is returned for the files after the failure. The CLI has no batch subcommand: `compress_files` is a library function with no CLI or GUI caller in the supplied code (a search of the supplied tree finds it used only by `tests/test_batch_compression.py`).

3.19 Command-Line Interface — `compressor/cli.py` (BL20)

- **Purpose.** The user-facing command interface; also the integration point that wires the pipeline, statistics, tree/table display, verification and performance measurement together.
- **Structure.** `build_parser()` builds an `argparse` parser (`prog="file-compression-tool"`); `main(argv=None)` parses arguments, dispatches to `_compress` / `_decompress` / `_verify` / `_history` through a handler dict, and returns `run_safely(handler, args)`. Full command/flag reference in §6.1.
- **`_compress`.** Wraps `pipeline.compress_file` in `measure(..., track_memory=args.time)`, prints `Compressed: <input> -> <output>`, prints each line of `format_statistics(result["stats"])`, and, only if requested, prints the Huffman code table (`--table`), the rendered tree (`--tree`), runs `assert_lossless` (`--verify`, printing `"Verification passed: the file restores exactly."`), and prints the performance line (`--time`).
- **`_decompress`.** Wraps `pipeline.decompress` in `measure`, prints `Decompressed: <input> -> <output> (<size>)`, and prints the performance line if `--time` was given.
- **`_verify`.** Raises `VerificationError("The restored data differs from the original file.")` if `verify_lossless` is false; otherwise prints `"OK: The compressed file restores the original exactly."`.
- **`_history`.** With `--clear`, clears the persistent history and prints `"History cleared."`. Otherwise prints each entry as `"<timestamp> | <operation> | <input> -> <output>"`, or `"No history available."` when empty.
- **Test status.** `cli.py` has **62% line / 50% branch** coverage — the lowest of any covered core module except `file_validation.py` (65%). The CLI's argument handling is therefore only partly exercised by the test suite; see §10.

3.20 Graphical Interface — `gui.py` (785 lines)

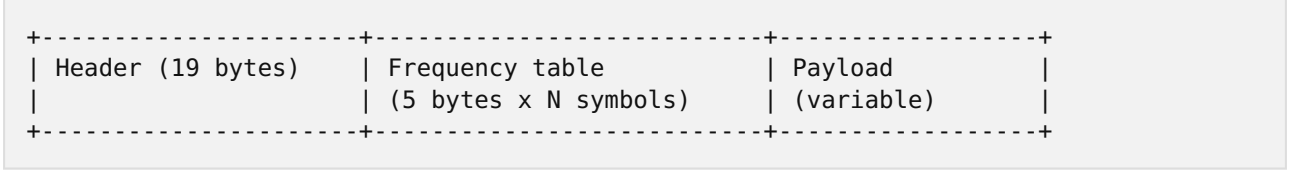
- **Purpose.** A desktop alternative to the CLI for users who prefer file pickers and on-screen results.
- **Structure.** `class FileCompressionGUI` (constructed with a Tk root; window 1080×750, minimum 850×620; dark theme, `ttk.Notebook` with the `clam` theme). Module function `main()` is the GUI entry point (run as `py gui.py` from the project root, per the module docstring).
- **Tabs and actions.** See §6.3. In brief: **Compress File** (choose file → compress → statistics), **Decompress File** (choose `.hfc` → restore), **Huffman Explorer** (code table + tree for a chosen file), **Statistics & Performance** (latest operation's timing/throughput/statistics), **Compression History** (session-only list, with Refresh and Clear Session History).
- **How it compresses.** `FileCompressionGUI.compress_file()` reads the file itself (`_read_file`, whole file), rejects an empty file with a warning, then calls `calculate_frequencies` → `build_huffman_tree` → `generate_huffman_codes` → `build_compressed_bytes` directly, times the four calls with `time.perf_counter()`, writes with its own `_write_new_file` (mode `"xb"`), computes statistics with `calculate_statistics(len(data), len(compressed))` (no `total_bits`), records the operation in its in-memory list via `_record_history`, and shows a message box. It does **not** call `pipeline.compress_file()`, `validate_input_file()`, `prepare_output_path()` / `write_output()`, `history.CompressionHistory`, `performance.measure()` or `verification.py`.
- **How it decompresses.** `decompress_file()` reads the file itself, calls `decompress_bytes(raw)` (which performs all format/CRC validation, §3.8), writes with `_write_new_file("xb")`, and records the operation in memory. Errors are shown with `messagebox.showerror(...)` including the exception text, and the status line is updated.
- **Error presentation.** Unlike the CLI, the GUI shows errors in message boxes (`showwarning` / `showerror` / `showinfo`) rather than stderr text and exit codes; there is no exit-code concept in the GUI. A cancelled save dialog sets the status to `"Compression cancelled."` / `"Decompression cancelled."` and returns without doing anything.
- **Test status.** `gui.py` is **not covered** by tests (not imported by them) — see §10.

---

4 Data Design

4.1 The `.hfc` File Format (HFC1)

A compressed file is three consecutive sections. There is no separate index, no footer and no trailing data.



Header — 19 bytes, big-endian, `struct` format `">4sQIBH"`.

Offset	Size	Format	Field	Meaning
0	4	`4s`	Magic	Always the bytes `HFC1`. Anything else is rejected on decompression.
4	8	`Q`	Original size	Length of the original file in bytes (unsigned 64-bit).
12	4	`I`	CRC32	`zlib.crc32(original data)` (unsigned 32-bit). Checked on decompression.
16	1	`B`	Padding	Unused bits in the last payload byte, 0–7.
17	2	`H`	Symbol count	Number of distinct byte values, `N` (unsigned 16-bit).

Frequency table — `N × 5` bytes, `struct` format `">BI"` per entry.

Offset	Size	Format	Field	Meaning
19 + 5·i	1	`B`	Symbol	One byte value, 0–255. Entries are in ascending symbol order.
20 + 5·i	4	`I`	Frequency	How many times that byte occurs in the original file. Must be > 0; duplicates are rejected.

The frequencies must sum to the original size recorded in the header; `decompress_bytes` enforces this before decoding.

Payload — variable length.

The original bytes encoded with their Huffman codes, packed most-significant-bit first into bytes (`encode_to_bytes`). The final byte is padded on the right with zero bits; the padding count is the header's padding field. The payload length in bytes is `(total_bits + 7) // 8`, where `total_bits = sum(freq[s] * len(codes[s]))`.

Total file size. `19 + 5·N + ceil(total_bits / 8)` bytes, where `N` is the number of distinct byte values. Worked check against the verified demo: a 10-byte input file produced a 42-byte `.hfc` file. (For the 10-byte `AAAABBBCCD` file: `N` = 4 distinct bytes, so the table is 20 bytes and the payload is 3 bytes — 19 + 20 + 3 = 42 bytes, which matches the measured output.)

Format properties and limits.

- **Self-describing.** Everything the decoder needs — original size, checksum, padding, and the full frequency table — is in the file. The Huffman tree and the codes are rebuilt, never stored.
- **Deterministic.** The same input always produces the same `.hfc` bytes, because frequencies are sorted, tree construction breaks ties by insertion order, and table entries are written in sorted-symbol order.
- **Size limit.** `MAX_FILE_SIZE = 0xFFFFFFFF` (4,294,967,295 bytes). `build_compressed_bytes` refuses larger inputs with `FileCreationError`. Note that the header's original-size field is 64-bit (`Q`), so the 4 GiB limit is imposed by the code constant, not by the header layout — see §10.

- **No streaming.** The whole compressed file is built in memory (`build_compressed_bytes` returns `bytes`) before it is written; the whole input is read into memory first (§3.10). See §10.

4.2 In-Memory Data Structures

Frequency dictionary. `{byte value (int, 0–255): count (int)}`, sorted by byte value. Produced by `calculate_frequencies`, consumed by `build_huffman_tree`, `generate_huffman_codes` (indirectly, via the tree), `build_compressed_bytes` and `build_huffman_code_table`.

Huffman tree. `HuffmanNode` objects linked by `left` / `right`. Leaf: `symbol` is an int, `left is None and right is None`. Internal node: `symbol is None`, `freq` is the sum of its children's frequencies. The tree is transient — built per operation, never serialised, never stored in the file.

Code dictionary. `{byte value: code string of "0"/"1"}`. Produced by `generate_huffman_codes`, consumed by `encode_to_bytes` (converted there to `{symbol: (int(code, 2), len(code))}` for bit-packing) and by `build_huffman_code_table`.

History JSON schema (`~/.file_compression_tool/history.json` — a JSON array of entry objects, at most 100, newest last):

```
[
  {
    "timestamp": "2026-11-06T12:00:00+00:00",
    "operation": "compression",
    "input_file": "notes.txt",
    "output_file": "notes.txt.hfc",
    "original_size": 90000,
    "compressed_size": 50914
  },
  {
    "timestamp": "2026-11-06T12:01:00+00:00",
    "operation": "decompression",
    "input_file": "notes.txt.hfc",
    "output_file": "notes_restored.txt",
    "restored_size": 90000
  }
]
```

Field rules (from `history.py`): `timestamp` is UTC ISO-8601; `operation` is exactly `"compression"` or `"decompression"`; `input_file` / `output_file` are strings; the extra detail keys are `original_size` and `compressed_size` for compression, `restored_size` for decompression. The example values above are illustrative of the schema only, except for the 90,000 / 50,914 pair, which is the verified demo measurement (§1.5). No real history file contents were in the evidence supplied.

Statistics dictionary. As specified in §3.11: `original_size`, `compressed_size`, `space_saved`, `space_saved_percent`, `compression_ratio`, `average_bits_per_symbol`.

Performance report. `PerformanceReport(seconds: float, peak_memory: int = 0)` (§3.15).

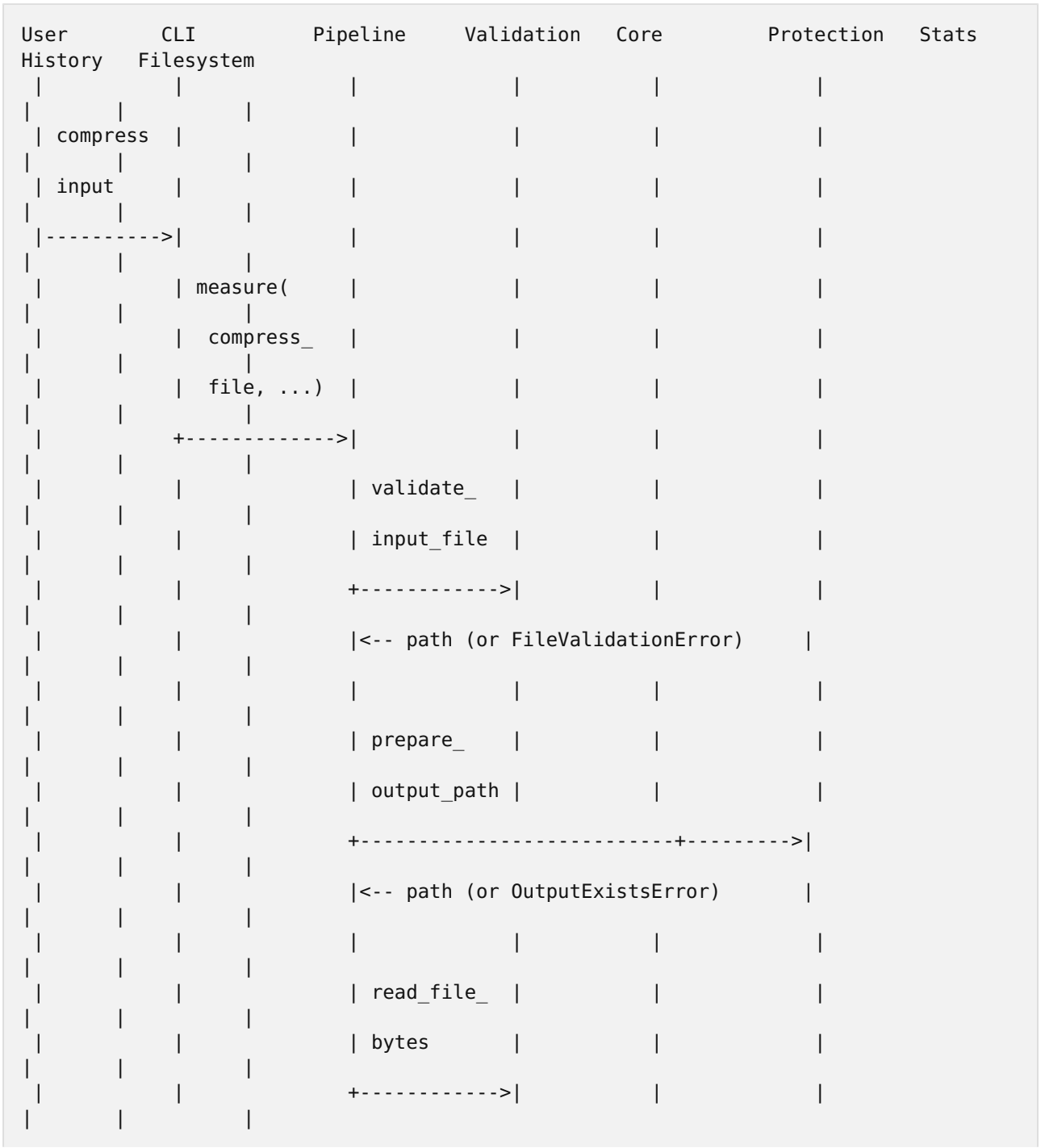
---

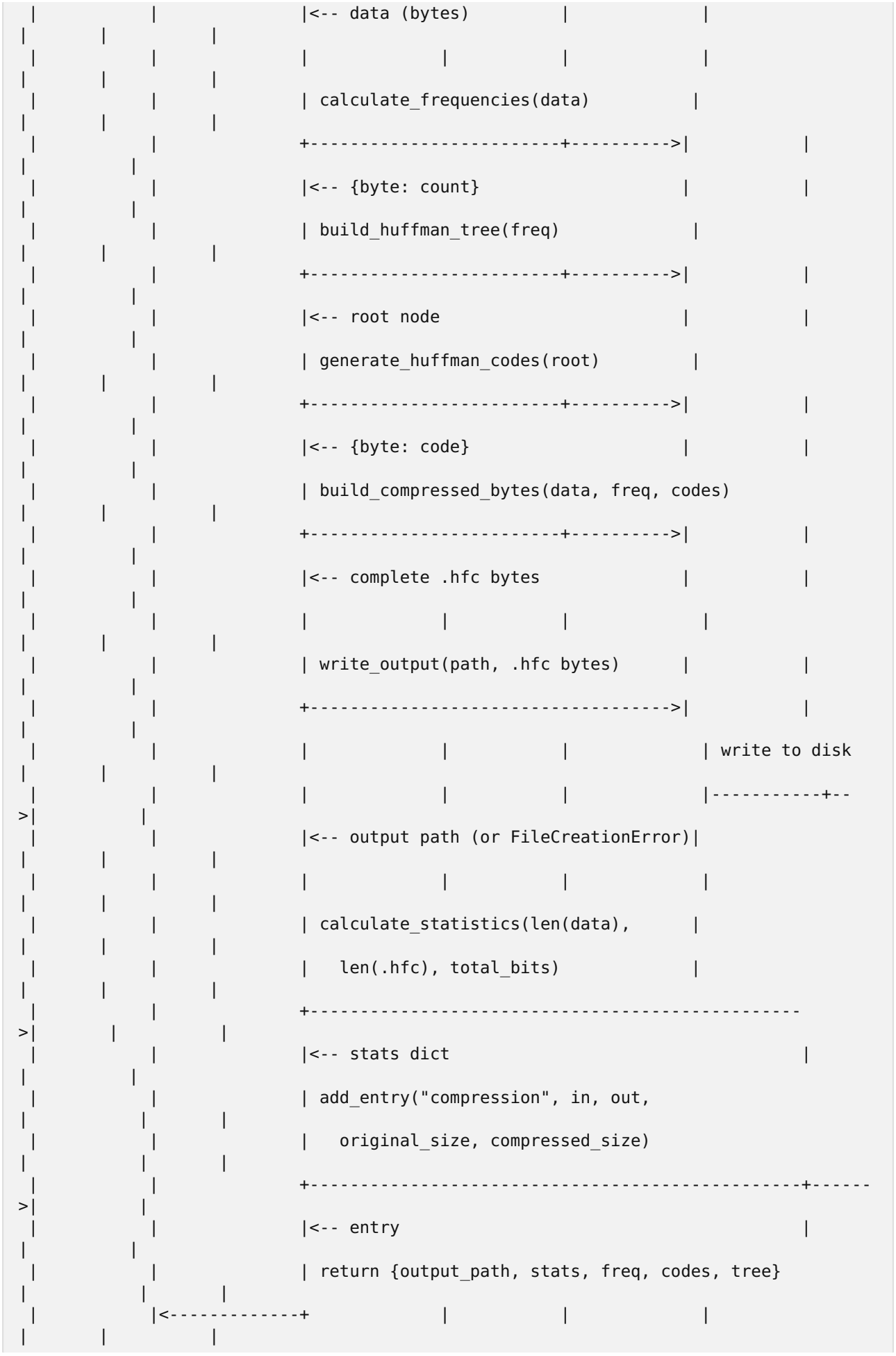
5 Sequence Diagrams

The diagrams below are UML sequence diagrams rendered as ASCII text (this SAD is delivered as Markdown; the same actor/object/lifeline/message content is what would be drawn as boxes and arrows in a diagramming tool). Lifelines run top to bottom; time runs downwards.

5.1 Compress a File End to End (CLI → Pipeline)

Actors / objects: `User`, `CLI (cli.py)`, `Pipeline (pipeline.py)`, `Validation (file_validation.py)`, `Core (frequency / huffman_tree / huffman_codes / file_format)`, `Protection (output_protection.py)`, `Stats (compression_stats.py)`, `History (history.py)`, `Filesystem`.




|<-- data (bytes)
| calculate_frequencies(data)
+----------+>|
|<-- {byte: count}
| build_huffman_tree(freq)
+----------+>|
|<-- root node
| generate_huffman_codes(root)
+----------+>|
|<-- {byte: code}
| build_compressed_bytes(data, freq, codes)
+----------+>|
|<-- complete .hfc bytes
| write_output(path, .hfc bytes)
+----------+>|
| write to disk
|<-- output path (or FileCreationError)|
| calculate_statistics(len(data),
| len(.hfc), total_bits)
+----------+>|
|<-- stats dict
| add_entry("compression", in, out,
| original_size, compressed_size)
+----------+>|
|<-- entry
| return {output_path, stats, freq, codes, tree}
|<----------+

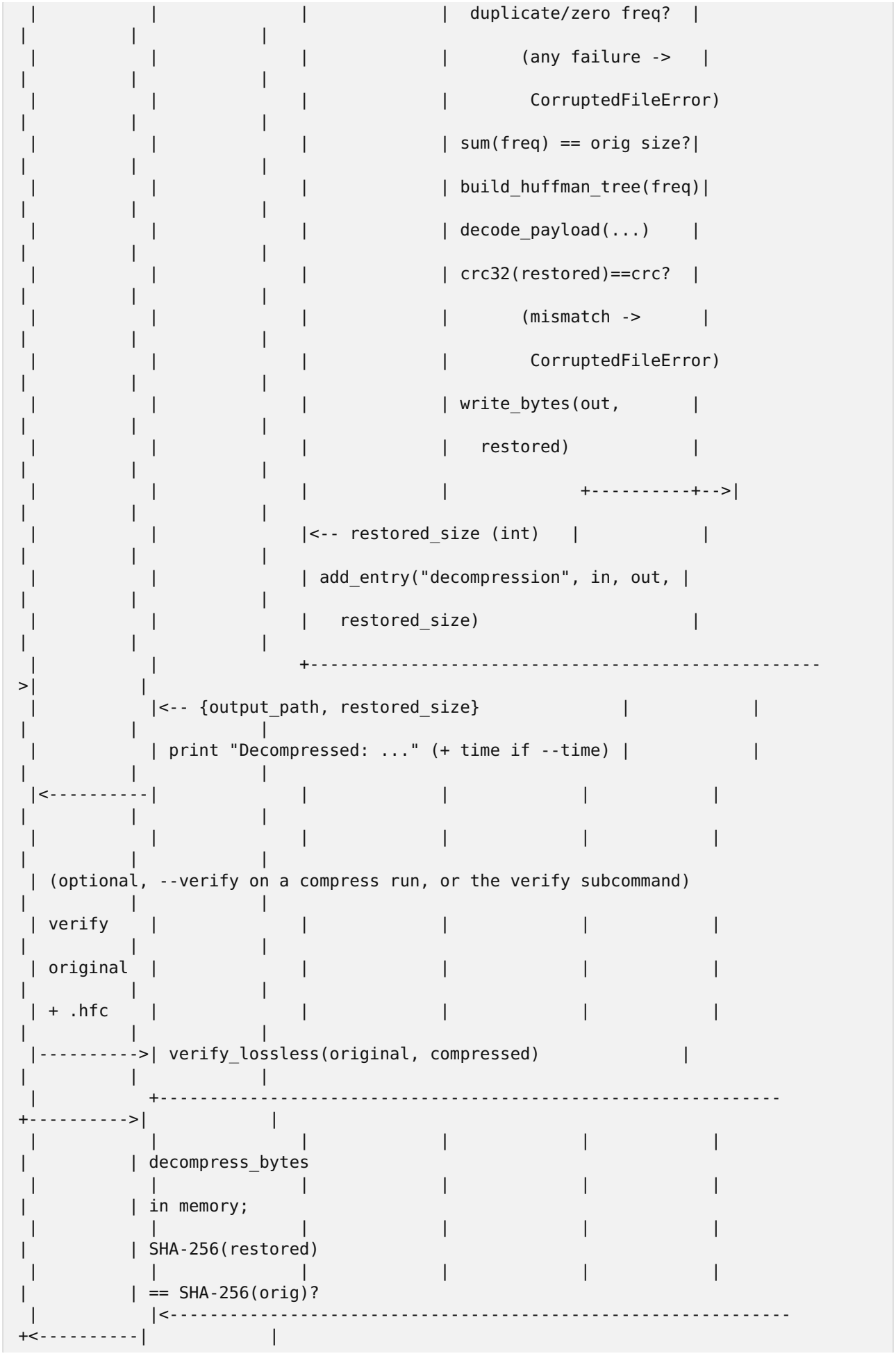
```
 |           | print stats (+ table / tree / verify / time, if flagged)
|       |          |
 |<----------|              |             |           |             |
|       |          |
```

Notes: (a) `measure()` in the CLI wraps the whole pipeline call, so the reported time includes validation, reading, tree building, encoding and writing. (b) `write_output` re-validates the output path internally (`prepare_output_path` is called twice for one compression — once in the pipeline, once inside `write_output`); this is harmless but redundant. (c) Any step raising a `CompressionError` aborts the sequence at that point; the CLI's `run_safely` converts it to a one-line stderr message and an exit code (§7) — no partial result is printed as success.

5.2 Decompress and Verify (CLI → Pipeline → Decompressor)

Actors / objects: `User`, `CLI`, `Pipeline`, `Validation`, `Decompressor (decompressor.py)`, `Protection`, `History`, `Filesystem`, `Verification (verification.py)`.

```
User        CLI          Pipeline     Validation   Decompressor  Protection  History
Filesystem  Verification
 |           |              |             |             |            |
|          |            |
 | decompress|              |             |             |            |
|          |            |
 | .hfc      |              |             |             |            |
|          |            |
 |---------->|              |             |             |            |
|          |            |
 |           | measure(     |             |             |            |
|          |            |
 |           |  decompress) |             |             |            |
|          |            |
 |           +------------->|             |             |            |
|          |            |
 |           |              | validate_input_file       |            |
|          |            |
 |           |              +------------>|             |            |
|          |            |
 |           |              | prepare_output_path       |            |
|          |            |
 |           |              +----------------------------------->|
|          |            |
 |           |              | decompress_file(in, out)  |            |
|          |            |
 |           |              +---------------------------------->|
|          |            |
 |           |              |             | read_file_bytes        |
|          |            |
 |           |              |             +------------>|          |
|          |            |
 |           |              |             |<-- raw .hfc bytes      |
|          |            |
 |           |              |             | parse_compressed(raw): |
|          |            |
 |           |              |             |  magic? padding?       |
|          |            |
 |           |              |             |  table complete?       |
|          |            |
```


duplicate/zero freq?
(any failure ->
CorruptedFileError)
sum(freq) == orig size?
build_huffman_tree(freq)
decode_payload(...)
crc32(restored)==crc?
(mismatch ->
CorruptedFileError)
write_bytes(out,
restored)
restored_size (int)
add_entry("decompression", in, out,
restored_size)
{output_path, restored_size}
print "Decompressed: ..." (+ time if --time)
(optional, --verify on a compress run, or the verify subcommand)
verify
original
+ .hfc
verify_lossless(original, compressed)
decompress_bytes
in memory;
SHA-256(restored)
== SHA-256(orig)?

```
 |           | "OK: ..." / VerificationError                |           |
|          |            |
 |<----------|              |             |             |           |
|          |            |
```

Note: verification re-decompresses the file in memory; it does not trust the file that was just written to disk. A file can therefore pass decompression (CRC32 matches) and still fail verification only if the original file given for comparison is not the one that was compressed — which is a user-input error, not a codec error.

5.3 Batch Compress (Library Function)

Actors / objects: `Caller`, `Batch (batch_compression.py)`, `Validation`, `Core`, `Protection`, `Filesystem`. There is no CLI or GUI caller for this function in the supplied code (see §3.18).

```
Caller      Batch        Validation   Core         Protection   Filesystem
 |           |              |           |             |           |
 | compress_ |              |           |             |           |
 | files([a, |              |           |             |           |
 |  b, ...]) |              |           |             |           |
 |---------->|              |           |             |           |
 |           | empty list? -> ValueError("Select at least one file to compress.")
 |           |              |           |             |           |
 |           | for each input_path:    |           |             |           |
 |           |   validate_input_file   |           |             |           |
 |           |  +--------------------->|           |             |           |
 |           |   read_file_bytes       |           |             |           |
 |           |  +--------------------->|           |             |           |
 |           |   calculate_frequencies / build_huffman_tree /                  |
 |           |   generate_huffman_codes / build_compressed_bytes               |
 |           |  +---------------------------------------->|           |           |
 |           |   output_path = default_compressed_name OR                    |
 |           |     output_dir / default_compressed_name                   |
 |           |   write_output(output_path, compressed)    |             |           |
 |           |  +-------------------------------------------------->|           |
 |           |  +---------------------------------------------------+-->|           |
 |           |   append {input_path, output_path,                    |             |
 |           |     original_size, compressed_size} to results       |             |
 |           |              |           |             |           |
 |<-- [result, result, ...] |           |             |           |
```

Notes: (a) No history entry and no statistics are produced for batch runs. (b) The loop is strictly sequential; one file's failure aborts the batch and propagates the exception to the caller — files already written are left in place.

---

6 API Design

6.1 CLI API

Run as `python main.py <subcommand> [options]` (entry point `main.py`; program name `file-compression-tool`). All subcommands are dispatched through `run_safely`, so every failure returns an exit code instead of a traceback (§7).

Subcommand	Arguments / flags	Behaviour
`compress`	`input` (positional, required); `-o` / `--output` (path, optional — default `<input>.hfc`); `--overwrite` (flag); `--table` (flag); `--tree` (flag); `--verify` (flag); `--time` (flag)	Compresses one file via `pipeline.compress_file`. Prints `Compressed: <input> -> <output>` and the formatted statistics. `--table` prints the Huffman code table; `--tree` prints the rendered tree; `--verify` runs `assert_lossless` and prints `Verification passed: the file restores exactly.`; `--time` prints the performance line (and enables memory tracking).
`decompress`	`input` (positional, required); `-o` / `--output` (path, optional — default `<input>` with `.hfc` stripped and `"_restored"` inserted before the extension); `--overwrite` (flag); `--time` (flag)	Decompresses one `.hfc` file via `pipeline.decompress`. Prints `Decompressed: <input> -> <output> (<size>)`. `--time` prints the performance line (and enables memory tracking).
`verify`	`original` (positional, required); `compressed` (positional, required)	Runs `verify_lossless`. Prints `OK: The compressed file restores the original exactly.` on success; raises `VerificationError` (exit code 6) on mismatch.
`history`	`--clear` (flag)	Without the flag, prints each stored entry as `<timestamp> | <operation> | <input> -> <output>`, or `No history available.`. With `--clear`, clears the persistent history and prints `History cleared.`

Worked examples (form only — file names are illustrative):

```
python main.py compress notes.txt
python main.py compress notes.txt -o notes.hfc --overwrite --table --tree --verify --
time
python main.py decompress notes.txt.hfc
python main.py decompress notes.txt.hfc -o notes_restored.txt --overwrite
python main.py verify notes.txt notes.txt.hfc
python main.py history
python main.py history --clear
```

6.2 Python Module API

The table lists the public functions and classes a caller (CLI, GUI, batch code, tests, or a future front end) should use. Internal helpers (`_sift_up`, `_sift_down`, `_read_file`, etc.) are not part of the public API and are not listed.

Module	Function / class	Signature (as coded)	Returns
`frequency`	`calculate_frequencies`	`(data: bytes)`	`{byte:` sorted by
`min_heap`	`MinHeap`	`()`, `.push(priority, item)`, `.pop()`, `.peek()`, `len()`, `.is_empty()`	`pop()` / `(priori`
`huffman_tree`	`HuffmanNode`		

Module	Function / class	Signature (as coded)	Returns
		`(freq, symbol=None, left=None, right=None)`, `.is_leaf()`	Node; `is` `bool`
`huffman_tree`	`build_huffman_tree`	`(freq: {byte: count})`	Root `Huf` `None` if
`huffman_tree`	`symbol_label`	`(symbol)`	Readable
`huffman_codes`	`generate_huffman_codes`	`(root)`	`{byte:` `string}` empty tre
`file_format`	`encode_to_bytes`	`(data, codes)`	`(payloa` `padding`
`file_format`	`build_compressed_bytes`	`(data, freq, codes)`	Complete `bytes`
`file_format`	`write_bytes`	`(path, data)`	`None`; re output or
`decompressor`	`parse_compressed`	`(raw: bytes)`	`(origin` `crc, pa` `freq, p`
`decompressor`	`decode_payload`	`(payload, root, original_size, padding)`	Restored
`decompressor`	`decompress_bytes`	`(raw: bytes)`	Restored (CRC32-ch
`decompressor`	`decompress_file`	`(input_path, output_path)`	`restore`
`file_validation`	`validate_input_file`	`(path, allow_empty=False)`	The path,
`file_validation`	`read_file_bytes`	`(path)`	File conte
`verification`	`file_hash`	`(path)`	SHA-256 string
`verification`	`files_identical`	`(path_a, path_b)`	`bool`
`verification`	`verify_lossless`	`(original_path, compressed_path)`	`bool`
`verification`	`assert_lossless`	`(original_path, compressed_path)`	`None`; ra `Verific` on misma
`compression_stats`	`calculate_statistics`	`(original_size, compressed_size, total_bits=None)`	Stats dict
`compression_stats`	`format_statistics`	`(statistics)`	Dict of di
`tree_visualizer`	`render_tree`	`(root, max_depth=8)`	Text diag
`huffman_code_table`	`build_huffman_code_table`	`(frequencies, codes)`	List of ro
`output_protection`	`default_compressed_name`	`(input_path)`	`str` (inp
`output_protection`	`default_decompressed_name`	`(compressed_path)`	`str` (wit

Module	Function / class	Signature (as coded)	Returns
`output_protection`	`prepare_output_path`	`(output_path, overwrite=False)`	Validated
`output_protection`	`write_output`	`(output_path, data, overwrite=False)`	Output p
`corruption`	`validate_structure`	`(original_size, padding, freq)`	`None`; ra `Corrupt`
`corruption`	`check_payload_length`	`(freq, codes, padding, payload)`	`None`; ra `Corrupt`
`corruption`	`check_checksum`	`(data, expected_crc)`	`None`; ra `Corrupt`
`performance`	`PerformanceReport`	Dataclass `(seconds: float, peak_memory: int = 0)`	—
`performance`	`measure`	`(func, *args, track_memory=False, **kwargs)`	`(result Perform`
`performance`	`format_performance`	`(report, size_bytes)`	`"Time: Speed:`
`history`	`CompressionHistory`	`(history_file=~/.file_compression_tool/history.json, max_entries=100)`; `.get_history()`, `.add_entry(opera input_file, output_file, **details)`, `.clear_history()`	List of en entry / `N`
`batch_compression`	`compress_files`	`(input_paths, output_dir=None, overwrite=False)`	List of pe dicts (§3.1
`pipeline`	`compress_file`	`(input_path, output_path=None, overwrite=False)`	`{"outpu "stats" "freque "codes"`
`pipeline`	`decompress`	`(input_path, output_path=None, overwrite=False)`	`{"outpu "restor`
`errors`	`CompressionError` + 5 subclasses	See §7	—
`error_handler`	`run_safely`	`(func, *args, **kwargs)`	Exit code
`error_handler`	`exit_code_for` / `format_error`	`(error)`	Exit code `"Error:`
`utils`	`format_size`	`(num_bytes)`	e.g. `2048 KB"`, sma B"
`cli`	`build_parser` / `main`	`()` / `(argv=None)`	Parser / e

API design rules the code follows (and one it breaks).

- Leaf functions take plain data (`bytes`, dicts, trees) and return plain data; they do not read `sys.argv`, open dialogs, or print. That is what makes them unit-testable.

- Errors are raised as typed exceptions (§7), never returned as error strings or sentinels (except `run_safely`, whose whole job is to convert exceptions into exit codes at the CLI boundary).
- **Broken rule (GUI):** the GUI duplicates orchestration and file handling instead of calling `pipeline.compress_file()` / `pipeline.decompress()` and `output_protection.write_output()`. Any future front end must call the pipeline, not re-implement it — see §10.

6.3 GUI API

The GUI has no command-line API and no network API. Its "API" is its five tabs and their controls (class `FileCompressionGUI` in `gui.py`; entry point `main()`; run as `py gui.py`).

Tab	Controls / actions	What happens
Compress File	Browse files (file picker); Compress Selected File; result text area	`choose_compress_file()` picks the input; `compress_file()` runs the leaf modules directly (§3.20), writes via `_write_new_file("xb")`, shows statistics and a completion message box, records the operation in the session history.
Decompress File	Choose .hfc File and Decompress; result text area	`decompress_file()` picks the `.hfc` file and the save location, calls `decompress_bytes(raw)`, writes the restored file exclusively, shows the result.
Huffman Explorer	Choose file; code-table + tree text area	`explore_file()` computes frequencies, tree and codes for the chosen file and displays `build_huffman_code_table(...)` rows and `render_tree(...)`.
Statistics & Performance	Refresh (implicit — updates after each operation); text area	`refresh_performance()` shows the latest operation's elapsed time, throughput and (for compression) statistics, with the note that a single-operation timing is not a controlled benchmark.
Compression History	Refresh History; Clear Session History; text area	`refresh_history()` / `clear_history()` show or clear the in-memory session list (`self.history`). History here is session-only and is lost when the application closes (§3.17).

---

7 Error Handling Protocols

7.1 Error Taxonomy

Every deliberate error is a `CompressionError` subclass (`compressor/errors.py`). The table gives the class, when it is raised, and the exact message pattern from the code.

Exception	Raised by / when	Example message (from code)
`FileValidationError`	`file_validation`: no/blank path; file not found; path is a directory; not readable; empty file (default); unreadable on read/hash	`File not found: '<path>'`; `'<path>' is a folder, not a file.`; `No permission to`

Exception	Raised by / when	Example message (from code)
		`read '<path>'.`; `'<path>' is empty - there is nothing to compress.`; `Could not read '<path>': ...`
`FileCreationError`	`file_format.build_compressed_bytes`: input over 4 GiB; `file_format.write_bytes`: cannot create/write; `output_protection`: output is a directory; parent directory does not exist; `write_output`: cannot write	`File is too large to compress (limit is 4 GB).`; `Could not create '<path>': ...`; `Could not write '<path>': ...`; `Output path refers to a directory: '<path>'`; `Output directory does not exist: '<parent>'`
`CorruptedFileError`	`decompressor` (and, in principle, `corruption`): short file; wrong magic; padding > 7; incomplete table; duplicate symbol; zero frequency; frequency sum mismatch; decode size mismatch; CRC32 mismatch; invalid empty/single-symbol payload	`File is too short to be a compressed file.`; `Not a valid compressed file (wrong signature).`; `Padding information is invalid.`; `The frequency table is incomplete.`; `The frequency table contains a duplicate symbol.`; `The frequency table contains an invalid frequency.`; `Frequency table does not match the original file size.`; `The data does not decode to the expected size.`; `Corrupted file: CRC32 checksum mismatch.`
`OutputExistsError`	`output_protection.prepare_output_path` / `write_output`: output exists and `overwrite` is false (including the exclusive-create race)	`Output file already exists: '<path>'. Choose another output path or enable overwrite.`
`VerificationError`	`verification.assert_lossless` / CLI `_verify`: restored data differs from the original	`Verification FAILED: the restored data differs from the`

Exception	Raised by / when	Example message (from code)
		`original.` (library); `The restored data differs from the original file.` (CLI `_verify`)
`CompressionError` (base)	Any of the above, caught generically; also the fallback in `exit_code_for`	— (the specific subclass message is used)

Non-`CompressionError` exceptions are not part of the taxonomy: `ValueError` / `TypeError` are raised for programming errors (empty batch list, mismatched code-table inputs, bad `max_entries`, non-boolean `overwrite`), `IndexError` for an empty heap, and `KeyboardInterrupt` / `MemoryError` are handled separately by `run_safely` (below).

7.2 Exit Codes (CLI)

`compressor/error_handler.py`, `EXIT_CODES` + `run_safely`:

Exit code	Meaning	Trigger
0	Success	Operation completed; also the code returned when `run_safely`'s function returns normally.
1	Other / unexpected error	Any `CompressionError` not in the table below; any non-`CompressionError` exception (`"Unexpected error: ..."`).
2	Validation error	`FileValidationError`
3	Creation error	`FileCreationError`
4	Corrupted file	`CorruptedFileError`
5	Output already exists	`OutputExistsError`
6	Verification failure	`VerificationError`
7	Out of memory	`MemoryError` (`"Error: not enough memory to process this file."`)
130	Cancelled by user	`KeyboardInterrupt` (`"Cancelled by the user."`)

7.3 User-Message Protocol

1. **One line, no traceback.** `run_safely` catches the exception and prints only `format_error(error)` — the string `"Error: <message>"` — to stderr. Internal details (stack traces, memory contents, data structures) are never shown to the user (SR-8).
2. **The message names the problem and, where possible, the remedy.** E.g. the `OutputExistsError` message tells the user to choose another path or enable overwrite; the `FileCreationError` for a missing parent directory names the directory.

3. **The GUI follows a different presentation protocol** (message boxes, §3.20) but the same content rule: it shows the exception text in the dialog and updates the status line (e.g. `"Decompression failed — file may be invalid or corrupted."`).
4. **Failures are never presented as successes.** A failed write does not leave a file that looks complete (§7.4); a failed verification raises instead of printing statistics as if the round trip had succeeded.

7.4 Partial-Output Cleanup Protocol

- `file_format.write_bytes`: if writing fails after the file was opened, it attempts `os.remove(path)` and then raises `FileCreationError`. A best-effort removal — if the removal itself fails, that failure is swallowed (`except OSError: pass`) and the creation error is still raised.
- `output_protection.write_output`: exclusive creation (`"xb"`) means a partially written file can only exist if the write itself was interrupted after creation; that path goes through the same `OSError → FileCreationError` conversion (note: `write_output` does **not** itself remove a partial file on write failure — removal on failure lives in `file_format.write_bytes`, which the pipeline/decompressor path uses, not in `write_output`. This asymmetry is a known inconsistency — see §10).
- `history.add_entry`: the history file is written to a `.tmp` sibling and atomically replaced, so a crash during the write cannot leave a half-written `history.json` (§3.17).
- The GUI's `_write_new_file` opens with `"xb"` and has no cleanup step of its own: if its write fails part-way, the partial file remains (the code opens with `"xb"` and has no cleanup step; no test covers this path).

7.5 Corrupted-File Detection Protocol (as actually executed)

On decompression, checks run in this order inside `decompressor.py` (not via `corruption.py` — see §3.14):

1. Length ≥ 19 bytes (header size).
2. Magic == `HFC1`.
3. Padding ≤ 7.
4. File long enough to contain the whole frequency table.
5. No duplicate symbol and no zero frequency in the table.
6. `sum(frequencies) == original_size` from the header.
7. Decode: the bit walk must end at the tree root and produce exactly `original_size` bytes (with the special empty-file and single-symbol cases in §3.8).
8. `CRC32(restored) == CRC32` from the header.

The first failure raises `CorruptedFileError` and stops the operation; no output file is written, because `decompress_file` only calls `write_bytes` after `decompress_bytes` has returned successfully. `test_corruption.py` exercises this protocol by flipping/truncating bytes and asserting `CorruptedFileError` (see §9).

---

8 Security Architecture

8.1 Security Objectives

From the checklist requirement (SAD security section: at least 2 objectives and 2 requirements) and the Security Requirements document (R4). The system's security objectives are:

1. **Integrity of user data.** Compression followed by decompression must return the original bytes exactly (SR-4); a damaged compressed file must be detected, never silently accepted (SR-2, SR-5).
2. **No loss or unintended modification of existing files.** Existing outputs must not be overwritten accidentally (SR-3); unrelated files must never be touched (SR-7); a failed operation must not leave a file that looks like a successful result (SR-10).
3. **Confidentiality by locality.** File contents are processed only on the user's own machine and are never transmitted anywhere (SR-9).
4. **Least information disclosure on failure.** Error messages must say what went wrong without exposing internals (SR-8).

8.2 Threats and Controls

Threat	What could go wrong	Control(s) in the design	Requirement
Non-existent / unreadable / empty input is processed	Crash, misleading "success", or an empty output presented as a result	`validate_input_file` before any processing; whole-file read wrapped in `FileValidationError`	SR-1, SR-5
Malformed or non-`.hfc` input is fed to the decoder	Crash, garbage output, or a wrong file silently "restored"	Header validation in `parse_compressed` (magic, length, padding, table completeness, duplicate/zero frequencies, frequency-sum check); decode-size check; CRC32 check	SR-2, SR-5
Corrupted payload (flipped / truncated / extra bytes)	Silent data corruption on restore	CRC32 over the original data stored in the header and checked in `decompress_bytes`; SHA-256 round-trip verification available via `verify_lossless` / `--verify`	SR-2, SR-4, SR-5
Accidental overwrite of an existing file	Loss of the user's existing data	Reject-by-default overwrite protection (`prepare_output_path`, `"xb"` exclusive creation); output path may not equal input path (GUI check; CLI default names never equal the input)	SR-3
Wrong output location / unrelated files modified	Data loss outside the requested operation	All writes go only to the user-specified (or default-derived) output path; no other paths are written; history is confined to `~/.file_compression_tool/history.json`	SR-7
Excessively large input		4 GiB limit in `build_compressed_bytes` (`MAX_FILE_SIZE`); `MemoryError` caught by	SR-6

Threat	What could go wrong	Control(s) in the design	Requirement
exhausts memory / disk	Crash or denial of service on the user's machine	`run_safely` → exit code 7 with a plain message. **Limit:** the limit is checked after the file has been read into memory, and there is no streaming path — see §10	
Error messages leak internals	Stack traces / memory contents shown to the user	`run_safely` prints only `"Error: <message>"` to stderr; no traceback is printed for expected errors	SR-8
File contents leave the machine	Confidential user data transmitted to a third party	No network code anywhere in the supplied tree: no sockets, no HTTP, no upload, no telemetry. Processing is local-only by construction	SR-9
Failed write leaves an incomplete file that looks valid	User trusts a truncated output	`file_format.write_bytes` removes a partial output on write failure (SR-10); history writes are atomic (`.tmp` + replace)	SR-10
History file is damaged or hand-edited	Crash on next start / history shown	`get_history()` returns `[]` for missing, unreadable, non-list or malformed JSON, and drops non-dict entries	SR-5 (robustness)

8.3 Control-to-Requirement Map (SR-1–SR-10)

SR	Control as implemented	Where	Status
SR-1 Validate input before compression/ decompression	`validate_input_file` (existence, directory, readability, non-empty) at the start of `pipeline.compress_file` / `pipeline.decompress` / `batch_compression.compress_files`	`file_validation.py`, `pipeline.py`, `batch_compression.py`	Implemented. GUI performs its c checks (`isfile` , non-empty) ins of calling `validate_input_fi` same outcome, different code (§
SR-2 Validate compressed files before decompressing	`parse_compressed` + frequency-sum + decode-size + CRC32 checks in `decompressor.py`	`decompressor.py`	Implemented — but via `decompressor.py` , **not** via `corruption.py` (§8.4).
SR-3 Prevent accidental overwriting	Reject-by-default + `"xb"` exclusive creation; `--overwrite` flag required to override	`output_protection.py`	Implemented on the rejection br The SRS wording also allows "rec confirmation"; the CLI does not p for confirmation — the flag is th explicit authorisation.
SR-4 Preserve integrity of the original	Lossless Huffman coding; CRC32 on every decompression; SHA-256 verification on demand	`decompressor.py` , `verification.py`	Implemented; verified by the de round trip (§1.5) and by `test_verification.py` / `test_integration.py` .
SR-5 Handle empty / corrupted / truncated	Empty input rejected at validation; corrupted input raises `CorruptedFileError` → one-line message + exit code 4, no crash, no output written	`file_validation.py` , `decompressor.py` , `error_handler.py`	Implemented for the decompres path. Empty-file *decompression* (c size 0) is handled as a special cas `decode_payload` (§3.8).

SR	Control as implemented	Where	Status
input gracefully			
SR-6 Validate sizes / limit memory use	`MAX_FILE_SIZE = 0xFFFFFFFF` enforced in `build_compressed_bytes`; `MemoryError` → exit code 7	`file_format.py`, `error_handler.py`	Partly implemented — see the §8 limit note and §10 (check happen after the whole-file read).
SR-7 Restrict operations to the specified paths	Only the input path is read; only the output path is written; no other file access	`pipeline.py`, `output_protection.py`, `history.py`	Implemented.
SR-8 Error messages without unnecessary internals	`run_safely` / `format_error`: `"Error: ..."` only, to stderr	`error_handler.py`	Implemented for the CLI. The GU shows the exception text in a dia which is the same message cont not a traceback.
SR-9 No transmission of file contents	No network code in the supplied tree; local-only by construction	Whole system	Implemented by absence — veri inspection of the module list (§2. there is no dependency that perf network I/O.
SR-10 Safely handle write failures	Partial-output removal in `file_format.write_bytes`; atomic history writes	`file_format.py`, `history.py`	Implemented for the pipeline/ decompressor path. `output_protection.write_o` and the GUI's `_write_new_fil` not remove partial outputs them — see §7.4 / §10.

8.4 Known Security Gap — `corruption.py` Is Not Wired In

`corruption.py` (BL15) provides the three checks the design intended for corrupted-file detection: `validate_structure`, `check_payload_length` and `check_checksum`. **None of them is called** by `pipeline.py`, `decompressor.py`, the CLI or the GUI, and the module has **0% coverage**. The checks that actually run are the equivalent checks re-implemented inside `decompressor.py` (§7.5). Consequences:

- `check_payload_length`'s exact-payload-length check — the one check that would catch **extra trailing bytes** appended to a valid `.hfc` file — is **not performed anywhere in the production code**. Whether trailing bytes are in practice harmless (the decoder reads only `total_bits` and ignores the rest) or a real integrity hole is a question for the test suite: `test_corruption.py` includes a `test_extra_bytes` case exercised through `decompress_bytes`, and it **passed** in the verified run (81/81): appending a byte raises `CorruptedFileError` ("The data does not decode to the expected size"). Extra bytes are therefore caught in practice by the decode-size check in `decompressor.py`, even though `check_payload_length` itself never runs.
- The duplication (checks in two places, one of them dead) is a maintenance risk: a future fix applied to `corruption.py` would silently have no effect, and a fix applied to `decompressor.py` would leave `corruption.py` inconsistent.

- **Fix (recommended, not yet done):** make `decompress_bytes` call `validate_structure` / `check_payload_length` / `check_checksum` from `corruption.py` (generating codes from the rebuilt tree for the length check), delete the duplicated inline checks, and add tests that import `corruption.py` directly. Until that is done, do not describe `corruption.py` as part of the active security path in the demo.

8.5 What This Architecture Deliberately Does Not Provide

- No authentication, authorisation or user accounts — the tool runs as the local user on their own files; there is nothing to log into.
- No encryption — `.hfc` files are not encrypted and the history file is plain JSON. Confidentiality rests entirely on local-only processing (SR-9), not on cryptography. Do not claim the format is encrypted.
- No audit log beyond the 100-entry operation history, which records file paths and sizes, not file contents.
- No protection against a maliciously crafted `.hfc` file beyond the validation in §7.5: the decoder is defensive against malformed structure, but it was not designed or tested as a hardened parser for hostile input (no fuzzing or hostile-input testing is in the evidence).

---

9 Traceability

9.1 Backlog → Component → Requirement → Test

FR-1–FR-10 and NFR-1–NFR-8 are from the SRS (R3); SR-1–SR-10 from the Security Requirements (R4). "—" means no numbered requirement in the SRS extract covers that item; the item is traced to its backlog item and test file instead, and the gap is flagged rather than filled with an invented FR number.

BL	Backlog item	Component(s)	FR / NFR / SR	Test file(s)
BL1	File Selection	CLI `input` argument; GUI file picker (`gui.py`)	FR-1	— (no dedicated test file; exercised indirectly through `test_integration.py`)
BL2	File Validation	`file_validation.py`	FR-1, FR-10; SR-1, SR-5	— (no dedicated test file in `tests/` ; `file_validation.py` coverage is 65% via other tests)
BL3	Frequency Analysis	`frequency.py`	FR-2	`test_integration.py` (and via tree/code tests); no dedicated `test_frequency.py` exists in `tests/`
BL4	Min Heap	`min_heap.py`	FR-3 (supports tree construction)	`test_huffman_tree.py` (heap exercised through tree building); no dedicated `test_min_heap.py` exists
BL5	Huffman Tree	`huffman_tree.py`	FR-3	`test_huffman_tree.py`

BL	Backlog item	Component(s)	FR / NFR / SR	Test file(s)
BL6	Huffman Codes	`huffman_codes.py`	FR-4	`test_huffman_codes.py`
BL7	Bitstream Encoding	`file_format.encode_to_bytes`; `decompressor.decode_payload`	FR-5, FR-7	`test_file_format.py`, `test_integration.py`
BL8	Compressed File Creation	`file_format.py`	FR-5, FR-6	`test_file_format.py`
BL9	File Decompression	`decompressor.py`	FR-7	`test_corruption.py`, `test_integration.py`, `test_verification.py`
BL10	Lossless Verification	`verification.py`	FR-10; SR-4	`test_verification.py`
BL11	Compression Statistics	`compression_stats.py`	FR-8	`test_compression_stats.py`
BL12	Huffman Tree Visualization	`tree_visualizer.py`	FR-9	`test_tree_visualizer.py`
BL13	Huffman Code Table	`huffman_code_table.py`	FR-9	`test_huffman_code_table.py`
BL14	Output & Overwrite Protection	`output_protection.py`	— (no FR); SR-3, SR-7	`test_output_protection.py`
BL15	Corrupted File Detection	`corruption.py` (unwired) + `decompressor.py` (actual checks)	FR-10; SR-2, SR-5	`test_corruption.py` — tests `decompressor.decompress_bytes`, **not** `corruption.py`
BL16	Performance Measurement	`performance.py`	NFR-1, NFR-6	`test_performance.py`
BL17	Error Handling	`error_handler.py`, `errors.py`	FR-10; SR-5, SR-8	`test_error_handler.py`
BL18	Batch Compression	`batch_compression.py`	FR-11 (SRS v2.0)	`test_batch_compression.py`
BL19	Compression History	`history.py`	FR-12 (SRS v2.0)	`test_history.py`
BL20	Integration & Final Testing	`pipeline.py`, `cli.py`, `main.py`	FR-5, FR-7 (end-to-end cycles); NFR-3, NFR-4	`test_integration.py`

9.2 Test Inventory (as verified)

- **Framework:** `unittest` (standard library), tests in `tests/`, with shared helpers in `tests/helpers.py` (e.g. `compress_bytes`, used by corruption/integration tests).
- **Result: 81 tests, all passed, 0.28–0.62 s.**

- **Coverage (compressor package):** line **83% (641 statements, 106 missed)**; branch **79% (226 branches, 35 partial)**.
- **Per-module coverage (verified):** `corruption.py` **0%**; `cli.py` **62% line / 50% branch**; `file_validation.py` **65%**; `gui.py` **not covered** (not imported by tests).
- **Test files (verified by listing `tests/`):** 14 `test_*.py` files (`test_batch_compression`, `test_compression_stats`, `test_corruption`, `test_error_handler`, `test_file_format`, `test_history`, `test_huffman_code_table`, `test_huffman_codes`, `test_huffman_tree`, `test_integration`, `test_output_protection`, `test_performance`, `test_tree_visualizer`, `test_verification`), 81 tests in total.
- **Traceability note.** The checklist requires the Test Plan to establish traceability back to the SRS and to contain 7–10 test cases covering FRs and NFRs. This SAD's traceability (§9.1) is the architecture-side half of that; the Test Plan document is a separate deliverable and is not reproduced here.

9.3 Requirement Coverage Summary

- FR-1–FR-10 are each implemented by at least one component (table §9.1). FR-8 (statistics display) is implemented in the CLI with `total_bits` and in the GUI without it — see §3.11.
- NFR-1 (reasonable time) is addressed by `performance.py` measurement and demonstrated by the 0.107 s demo (§1.5); no absolute time limit is stated in the SRS; "reasonable" is measured by the reported elapsed time (0.107 s for 90,000 bytes in the demo).
- NFR-2 (compressed output smaller, for suitable files) holds for repetitive data (43.43% saved in the demo) and fails by design for tiny files (10 bytes → 42 bytes) — see §10.
- NFR-3 (lossless round trip) is enforced by CRC32 + SHA-256 verification.
- NFR-5 (simple interface) is the CLI's four subcommands and the GUI's five tabs.
- NFR-7 (modular implementation) is the layered design of §2.
- NFR-8 (portability) — Python standard library only, no third-party dependencies in the core — is untested as a cross-platform claim: the only recorded run is Ubuntu 24.04.5 LTS, Python 3.12.3.

---

10 Deviations & Known Limitations

Per the FAQ document (R1), deviations from the initial SRS/SAD are acceptable **if documented and explained in the demo**. This section is that record. Each item states the deviation, its effect, and what should be done or said about it.

10.1 GUI does not use the pipeline (deviation — fix recommended)

- **What:** `gui.py` calls the leaf modules (`calculate_frequencies`, `build_huffman_tree`, `generate_huffman_codes`, `build_compressed_bytes`, `decompress_bytes`) directly and re-implements file reading, writing (`"xb"`), timing and history in its own methods. It never calls `pipeline.compress_file()` / `pipeline.decompress()`, `validate_input_file()`, `output_protection.write_output()` or `history.CompressionHistory`.
- **Effect:** Two compression paths that can drift apart. They already have: the GUI omits `total_bits` from its statistics (so average bits per symbol is `"N/A"` in the GUI), keeps

history only in memory, and does not run `verification.py`. A bug fixed in the pipeline would not be fixed in the GUI.
- **Demo line:** "The GUI is a second front end over the same leaf modules, but it does not yet go through the pipeline — that is a known inconsistency we would fix by making the GUI call `pipeline.compress_file()`."
- **Fix:** Route both GUI actions through `pipeline.py`; keep the GUI responsible only for picking files and displaying results.

10.2 `corruption.py` is not wired into any production path (deviation — fix recommended)

- **What:** BL15's three functions exist, are correct as written, and are never called; the module has 0% coverage (§3.14, §8.4).
- **Effect:** The exact-payload-length check (`check_payload_length`) never runs. Corruption detection works anyway through the checks in `decompressor.py` — including for extra trailing bytes, which the decode-size check catches (`test_extra_bytes` passed). The dead code is still a maintenance risk: the same checks exist in two places and can drift apart.
- **Fix:** Call the three functions from `decompress_bytes` (building codes from the rebuilt tree for the length check), remove the duplicated inline checks, and add direct unit tests for `corruption.py`.

10.3 CLI and GUI are only partly tested / untested

- **What:** `cli.py` 62% line / 50% branch coverage; `gui.py` 0% (not imported by tests); `file_validation.py` 65%.
- **Effect:** Argument handling, the `--table` / `--tree` / `--verify` / `--time` branches and the whole GUI are exercised only by manual demo, not by the automated suite. The 81 passing tests prove the core algorithm, not the front ends.
- **Demo line:** Quote the coverage figures exactly as in §1.5; do not claim "fully tested".

10.4 Batch compression has no front-end caller and no history/statistics

- **What:** `compress_files` (BL18) is a library function with no CLI subcommand and no GUI button in the supplied code; it records no history and computes no statistics, and a mid-batch failure aborts the batch with earlier files left compressed (§3.18).
- **Effect:** The feature exists in code but is not reachable from the product as a user would run it.
- No caller exists in the supplied tree; `compress_files` is used only by its tests.

10.5 Small files grow when compressed (limitation, by design)

- **What:** The 19-byte header plus 5 bytes per distinct symbol is fixed overhead. A 10-byte file became 42 bytes.
- **Effect:** NFR-2 (output smaller than input) cannot hold for very small or already-random files. This is inherent to storing the frequency table in the file, not a bug.
- **Demo line:** "Compression pays off on larger, repetitive files — 90,000 bytes to 50,914 in the demo — and the format overhead dominates on tiny files."

10.6 4 GiB size limit, checked late; whole-file in-memory processing (limitation)

- **What:** `MAX_FILE_SIZE = 0xFFFFFFFF` is enforced in `build_compressed_bytes`, which runs *after* `read_file_bytes` has already loaded the whole file into memory. The pipeline holds the input, the frequency table, the tree, the codes and the complete compressed output in memory at once. Only `file_hash` streams (1 MiB chunks).
- **Effect:** A file near the limit can exhaust memory before the limit is ever checked (SR-6 is therefore only partly met); very large files cannot be processed in a streaming fashion at all.
- **Fix (future work, not in scope for this version):** check the file size in `validate_input_file` (via `os.path.getsize`, which it already calls for the empty check) before reading, and/or add a streaming encoder. Do not present the current design as streaming.

10.7 Miscellaneous inconsistencies to be aware of in the viva

- `write_output` (output protection) does not remove a partial file on write failure; `write_bytes` (file format) does (§7.4). The pipeline's compress path uses `write_output`; its decompress path uses `write_bytes` via `decompress_file`.
- The GUI's `_write_new_file` has no partial-output cleanup of its own (§7.4).
- Two symbol-labelling helpers exist (`huffman_tree.symbol_label` and the private labellers in `tree_visualizer.py`) with slightly different rules; they are not shared.
- `prepare_output_path` is called twice on the CLI compress path (once in the pipeline, once inside `write_output`) — harmless, redundant.
- History is split: persistent JSON history for the CLI, session-only in-memory history for the GUI (§3.17).
- The testing reports named in the checklist (unit, system-validation, code-coverage and branch-coverage) are delivered as `Testing_Reports.md` / PDF with this set. The Sprint 1 (1-minute) and Sprint 2 (2-minute) videos were not in the evidence supplied and must be added from the real repository, not invented.

Appendix A — Backlog Summary

Source: GitHub Projects board `File_Compression_Tool_BLs`, Backlog view (screenshot supplied). All 20 items are `Done`. Story points total **99**.

BL	Title	Assignee	Story points	Sprint
BL1	File Selection	KeerthanaKumaresh1	3	Sprint (pract)
BL2	File Validation	KeerthanaKumaresh1	3	Sprint 1
BL3	Frequency Analysis	KeerthanaKumaresh1	5	Sprint 1
BL4	Min Heap	KeerthanaKumaresh1	5	Sprint 1
BL5	Huffman Tree	rishi2908	8	Sprint 1
BL6	Huffman Codes	rishi2908	5	Sprint 1
BL7	Bitstream Encoding	rishi2908	5	Sprint (pract)

BL	Title	Assignee	Story points	Sprint
BL8	Compressed File Creation	khushi309	5	Sprint 1
BL9	File Decompression	KeerthanaKumaresh1	5	Sprint 2
BL10	Lossless Verification	KeerthanaKumaresh1	3	Sprint 2
BL11	Compression Statistics	rishi2908	3	Sprint 2
BL12	Huffman Tree Visualization	khushi309	5	Sprint 1
BL13	Huffman Code Table	rishi2908	3	Sprint 1
BL14	Output & Overwrite Protection	rishi2908	5	Sprint 2
BL15	Corrupted File Detection	KeerthanaKumaresh1	5	Sprint 2
BL16	Performance Measurement	khushi309	5	Sprint 2
BL17	Error Handling	khushi309	5	Sprint 1
BL18	Batch Compression	khushi309	8	Sprint (pract)
BL19	Compression History	rishi2908	5	Sprint 2
BL20	Integration & Final Testing	khushi309	8	Sprint 2

Sprint totals: Sprint (pract) — BL1, BL7, BL18 = 16 points. Sprint 1 — BL2, BL3, BL4, BL5, BL6, BL8, BL12, BL13, BL17 = 44 points. Sprint 2 — BL9, BL10, BL11, BL14, BL15, BL16, BL19, BL20 = 39 points. Total 16 + 44 + 39 = **99 points**.

Workload by assignee (story points): KeerthanaKumaresh1 — BL1, BL2, BL3, BL4, BL9, BL10, BL15 = **29 points**. rishi2908 — BL5, BL6, BL7, BL11, BL13, BL14, BL19 = **34 points**. khushi309 — BL8, BL12, BL16, BL17, BL18, BL20 = **36 points**. Total 29 + 34 + 36 = **99 points**.

Notes. Assignees are GitHub usernames exactly as shown in the backlog screenshot; no full names are stated here except Keerthana Kumaresh, the author named in `README.md`. The backlog screenshot shows no Epic, Priority or Size values filled in for these items and no linked pull requests; those columns are therefore not reported here. PR numbers, GitHub Actions runs and video links were not in the evidence supplied; check them in the real repository (see the note on the title page).

End of SAD Specification v1.0 — File Compression Tool.